基于 Keras 构建的生成对抗网络 GAN *

李宇豪 †

(中山大学物理学院，广东 广州，510275)

2022 年 12 月 6 日

摘要

组会中的一个练习：真实数据为 1000 维的向量，其中 10% 是 +1，90% 是 −1；生成器和判别器均由一层网络构成；使用 GAN 将一个白噪声数据训练至逼近真实数据。本文首先回顾了 GAN 的基本原理，然后使用 Keras 搭建了一个简单的 GAN 进行训练，最终可以使得生成数据中 +1 的个数接近真实数据中的个数（100 个）。

1 GAN 的基本原理

生成对抗网络的模型结构如图 1 所示。网络分为生成器和判别器两个部分。生成器根据随机变量 $\boldsymbol{z}$ 生成假数据 $G(\boldsymbol{z})$（称为“生成数据”）；判别器对生成数据和真实数据进行判断，给出输入数据来着自真实分布的概率 D，并反馈给生成器进行下一步训练。模型达到最优的情况时，D 无法判别输入数据是来自真实数据 $\boldsymbol{x}$ 还是生成数据 $G(\boldsymbol{z})$，即 D 每次的输出概率值都为 1/2，此时生成数据无限接近于真实数据。

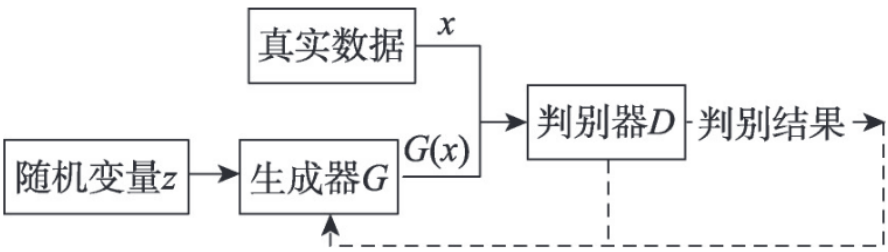


图 1: GAN 模型结构示意图

*PMI 组会作业

†Email:liyh536@mail2.sysu.edu.cn

1.1 判别网络

判别器的核心目标是区分出一个样本 $\boldsymbol{x}$ 是来自于真实分布 $p_r(\boldsymbol{x})$ 还是来自于生成模型 $p_\theta(\boldsymbol{x})$，实际上是一个二分类器。

对于一个样本 $\{\boldsymbol{x}, y\}$，标签设置为 $y \in \{1, 0\}$，分别表示 $\boldsymbol{x}$ 来自 $p_r(\boldsymbol{x})$ 和 $p_\theta(\boldsymbol{x})$。对于二分类问题，采用交叉熵损失函数，则判别网络的目标函数为：

$$\min_{\phi} \ -(\mathbb{E}_{\boldsymbol{x}}[y \log p(y=1 \mid \boldsymbol{x}) + (1-y) \log p(y=0 \mid \boldsymbol{x})]) \tag{1}$$

若假设分布 $p(\boldsymbol{x})$ 是由真实分布 $p_r(\boldsymbol{x})$ 和生成模型 $p_\theta(\boldsymbol{x})$ 均匀混合而成，即

$$p(\boldsymbol{x}) = \frac{1}{2}\left(p_r(\boldsymbol{x}) + p_\theta(\boldsymbol{x})\right) \tag{2}$$

那么 Eq.(1) 可以写为

$$\max_{\phi} \ \mathbb{E}_{\boldsymbol{x} \sim p_r(\boldsymbol{x})}[\log D(\boldsymbol{x}; \phi)] + \mathbb{E}_{\boldsymbol{z} \sim p(z)}[\log(1 - D(G(\boldsymbol{z}; \theta); \phi))] \tag{3}$$

其中，θ 是生成网络的参数，ϕ 是判别网络的参数。它的含义是：判别网络的损失由将真实数据判定为真和将生成数据判定为假两部分构成。

1.2 生成网络

生成网络的目标是尽量生成判别网络无法区分的数据，即让判别网络将生成数据判别为真实数据。其目标函数为

$$\min_{\theta} \left(\mathbb{E}_{\boldsymbol{z} \sim p(z)}[\log(1 - D(G(\boldsymbol{z}; \theta); \phi))]\right) \tag{4}$$

注意到函数 $\log(x), x \in (0, 1)$ 在 x 接近 1 时的梯度要比接近 0 时的梯度小很多。当判别网络 D 以很高的概率认为生成数据是假数据时，$(1 - D(G(\boldsymbol{z}; \theta); \phi)) \to 1$，此时目标函数关于 θ 的梯度很小，不利于优化，因此将 Eq.(4) 改写为

$$\max_{\theta} \left(\mathbb{E}_{\boldsymbol{z} \sim p(z)}[\log D(G(\boldsymbol{z}; \theta); \phi)]\right) \tag{5}$$

1.3 网络的训练

GAN 的思想来自于博弈论中的二人零和博弈 [13]，生成器和判别器可以看成是博弈中的两个玩家。在模型训练的过程中生成器和判别器会各自更新自身的参数使得损失最小，通过不断迭代优化，最终达到一个纳什均衡状态。

和单目标的优化任务相比，GAN 的两个网络的优化目标刚好相反。因此训练往往不太稳定. 一般情况下，需要平衡两个网络的能力。对于判别网络来说，一开始的判别能力不能太强，否则难以提升生成网络的能力。但是，判别网络的判别能力也不能太弱，否则针对它训练的生成网络也不会太好。在训练时需要使用一些技巧，使得在每次迭代中，判别网络比生成网络的能力强一些，但又不能强太多。

GAN 的训练流程如算法 1 所示. 每次迭代时，判别网络更新 K 次而生成网络更新一次，即首先要保证判别网络足够强才能开始训练生成网络，其中 K 是一个超参数。

算法 1: 生成对抗网络的训练过程

输入:

训练集 $\mathcal{D}$;
对抗训练迭代次数 T;
每次判别网络的训练迭代次数 K;
小批量样本数量 M;

输出:

生成网络 $G(\boldsymbol{z};\theta)$

for $t \leftarrow 1$ **to** T **do**
  // 训练判别网络 $D(\boldsymbol{x};\phi)$
  for $k \leftarrow 1$ **to** K **do**
    // 采集小批量训练样本
    从训练集 $\mathcal{D}$ 中采集 M 个样本 $\left\{\boldsymbol{x}^{(m)}\right\}, 1 \le m \le M$;
    从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\left\{\boldsymbol{z}^{(m)}\right\}, 1 \le m \le M$;
    使用随机梯度上升更新 ϕ, 梯度为

$$\frac{\partial}{\partial \phi}\left[\frac{1}{M}\sum_{m=1}^{M}\left(\log D\left(x^{(m)};\phi\right)+\log\left(1-D\left(G\left(z^{(m)};\theta\right);\phi\right)\right)\right)\right]$$

  end
  // 训练生成网络 $G(\boldsymbol{z};\theta)$
  从分布 $\mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 中采集 M 个样本 $\left\{\boldsymbol{z}^{(m)}\right\}, 1 \le m \le M$;
  使用随机梯度上升更新 θ, 梯度为

$$\frac{\partial}{\partial \theta}\left[\frac{1}{M}\sum_{m=1}^{M} D\left(G\left(z^{(m)};\theta\right),\phi\right)\right]$$

end

2 基于 Keras 搭建一个 GAN

使用 Keras 框架搭建一个简单的生成对抗网络，所需要使用的包如下：

```
from numpy import arange, expand_dims, array
from numpy.random import randn
from keras.models import Sequential
from keras.layers.core import Dense, Activation
from keras.optimizers import Adam, SGD
from matplotlib import pyplot as plt
```

首先搭建生成器和判别器的架构。本文中生成器和判别器均使用一层全连接层搭建，生成器将一个 100 维的高斯噪声向量映射到 1000 维，判别器将 1000 维的输入向量映射到 1 维，用于进行二分类。

```
def generator_model():
    # 搭建生成器
    model = Sequential()
    # 使用一个全连接层，输入为100维，输出为1000维
    model.add(Dense(1000, input_dim=100))
    # 激活函数使用tanh函数
    model.add(Activation('tanh'))
    return model

def discriminator_model():
    # 搭建判别器
    model = Sequential()
    # 使用一个全连接层，输入为1000维，输出为1维
    model.add(Dense(1, input_dim=1000))
    # 激活函数使用sigmoid函数
    model.add(Activation('sigmoid'))
    return model
```

在训练过程中需要交替训练生成器和判别器，因此考虑将上面搭建的生成器和判别器模型拼接在一起。

```
def generator_containing_discriminator(g, d):
    # 将生成器架构和判别器架构组拼接成一个大的神经网络，用于判别生成数据
    model = Sequential()
    # 先添加生成器g，再固定判别器d，在给定判别器d的情况下训练生成器g，
    # 将生成数据投入到判别器进行辨别从而优化生成器
    model.add(g)
    d.trainable = False
    model.add(d)
    return model
```

接下来定义训练过程

```
def train():
```

首先生成真实数据，一个 1000 维的向量，其中 10% 是 +1，90% 是 −1，并且设置真实数据的标签为 $y = 1$，生成数据的标签为 $y = 2$。

```
    # 真实数据
    real_data = arange(1000)
    real_data[real_data < 100] = 1
    real_data[real_data != 1] = -1
    # 可以用下面这行代码打乱真实数据的顺序
    # shuffle(real_data)

    # 设置真实数据和生成数据标签
    real_label = array([1])
    gen_label = array([0])
```

然后初始化生成器、判别器和按顺序拼接在一起的生成判别网络，并定义优化算法和超参数。

```
    # 将定义好的模型架构赋值给特定的变量
    d = discriminator_model()
    g = generator_model()
    d_on_g = generator_containing_discriminator(g, d)

    # 定义各个模型所使用的优化算法及超参数
    d_optim = Adam(learning_rate=0.0007)
    g_optim = Adam(learning_rate=0.0007)
    # 可以用以下代码使用随机梯度下降法
    # d_optim = SGD(learning_rate=0.0005, momentum=0.9, nesterov=True)
    # g_optim = SGD(learning_rate=0.0009, momentum=0.9, nesterov=True)

    # 编译网络，设置损失函数和优化算法，其中损失函数均为交叉熵损失函数
    g.compile(loss='binary_crossentropy', optimizer="Adam")
    d_on_g.compile(loss='binary_crossentropy', optimizer=g_optim)
    d.trainable = True
    d.compile(loss='binary_crossentropy', optimizer=d_optim)
```

另外，为了处理结果方便，创建 3 个数组分别存储迭代过程中生成器和判别器的损失，以及每次迭代后生成数据中 +1 的个数（即网络的训练目标）。

```
    # 绘图
    g_loss_list = []
    d_loss_list = []
    g_data_list = []
```

下面开始训练网络，整个网络的训练过程分为两部分。首先，生成器根据一个 100 维的高斯噪声进行推断，得到生成数据，然后计算判别器的损失（这个损失由生成数据的损失和真实数据的损失两部分构成，见 Eq.(3)）。接下来将高斯噪声和真实数据的标签（这是生成器的优化目标所希望的）放入 d_on_g 中，生成器生成数据后，放入已经固定的判别器中计算生成器的损失。这是一个完整的训练流程，在下面的代码中，首先进行 100 次这样的训练，分别记录第 100 次时生成器和判别器的损失以及生成数据中 +1 的个数，然后将这个过程视作一个 epoch，进行 100 个 epoch 的训练。这样的好处将在下一节中见到。

```
# 对抗网络迭代次数
for T in range(100):
    print("Epoch is", T)

    # 一个训练流程的迭代次数
    for K in range(100):
        # 随机生成100个标准高斯噪声
        noise = randn(100)
        # 使用生成器对随机噪声进行推断
        gen_data = g.predict(expand_dims(noise,axis=0),
                             verbose=0).reshape(-1)
        gen_data[gen_data > 0] = 1
        gen_data[gen_data < 0] = -1
        # 判别器的损失
        d_gen_loss = d.train_on_batch(gen_data.reshape(1,-1),
                                      gen_label.reshape(1,-1))
        d_real_loss = d.train_on_batch(real_data.reshape(1,-1),
                                       real_label.reshape(1,-1))
        d_loss = d_gen_loss + d_real_loss

        # 随机生成100个标准高斯噪声
        noise = randn(100)
        # 固定判别器
        d.trainable = False
        # 对生成器进行训练并计算损失
        g_loss = d_on_g.train_on_batch(noise.reshape(1,-1),
                                       real_label.reshape(1,-1))

        # 令判别器可训练
        d.trainable = True

        # 统计生成数据
        gen_data_finally = g.predict(expand_dims(noise,axis=0),
```

```
                                        verbose=0)
            gen_data_finally[gen_data_finally > 0] = 1
            gen_data_finally[gen_data_finally < 0] = -1
            result = gen_data_finally[gen_data_finally == 1].size
            print("第%d次迭代，判别网络损失为：%f，\
                  生成网络损失为：%f" % (K, d_loss, g_loss))
            print("结果统计：生成数据中1的个数", result)
            if K == 99:
                d_loss_list.append(d_loss)
                g_loss_list.append(g_loss)
                g_data_list.append(result)

    # 这里是整个 def train ():函数的返回值
    return g_loss_list, d_loss_list, g_data_list
```

训练结果的处理：

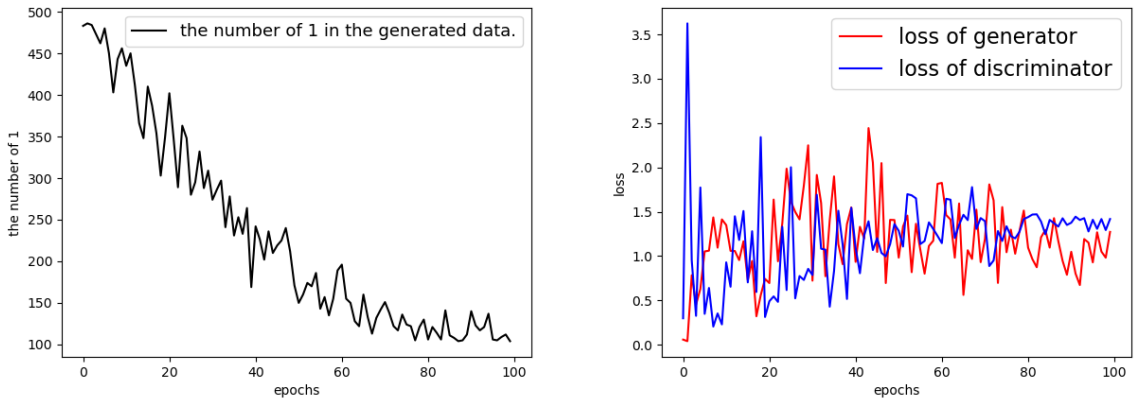
```
if __name__ == "__main__":
    g_loss_list, d_loss_list, g_data_list = train()

    plt.plot(range(len(g_loss_list)), g_loss_list, linewidth=1.5,
             c='red', label='loss of generator')
    plt.plot(range(len(d_loss_list)), d_loss_list, linewidth=1.5,
             c='blue', label='loss of discriminator')
    plt.xlabel('epochs')
    plt.ylabel('loss')
    plt.legend(loc='best', fontsize=16)
    plt.show()
    plt.clf()
    plt.plot(range(len(g_data_list)), g_data_list,
             c='black', label='the number of 1 in the generated data.')
    plt.legend(loc='best', fontsize=13)
    plt.xlabel('epochs')
    plt.ylabel('the number of 1')
    plt.show()
```

完整代码已经上传 GitHub[1]。

3 训练结果

设置学习率为 0.0007，使用 Adam 优化器，epoch 为 100，bitch 为 100，训练用时约为 30 分钟，结果如图 2 所示。

[1] https://github.com/lyhsysu/my_PMI_code/blob/main/GAN_by_Keras.py



(a) (b)

图 2: 训练结果 (a) 生成数据中 1 的个数随训练次数的变化 (b) 生成器和判别器的损失随训练次数的变化，其中红线是生成器的损失，蓝线是判别器的损失。

可以看到，训练过程中，生成模型的损失和判别模型的损失在一定范围内交替上升与下降，这相当于一个二者进行博弈的过程，+1 的个数也是在波动中总体呈下降趋势，最终在 95 个 epoch 的训练后降到 10% 附近。

这个网络在训练时需要经过大量迭代，这也是在搭建网络时分成 100 个 epoch 和 100 个 bitch 的原因，实际上这种分法也相当于直接进行 10000 次训练，每 100 次训练进行一次采样。如果不进行间隔采用，得到的结果如图 3 所示。这也正是 GAN 模型训练不稳定的体现。或许可以尝试使用 BEGAN 等变体解决这个问题。

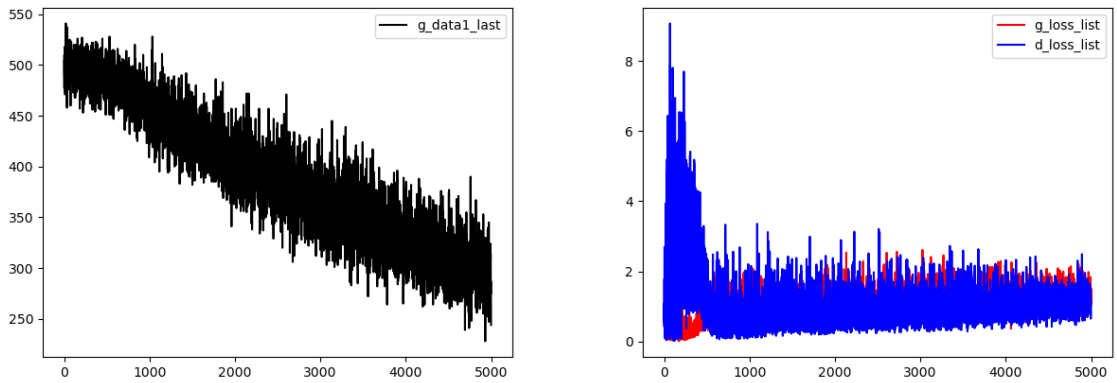


图 3: 不进行间隔采样时绘制的 GAN 训练结果